

Clue Selection Under Noise

Codenames spymaster, baseline model

1 Introduction

A clue is a clue word together with a number k . Whenever we give one, our teammate ranks the unrevealed board words by how well each fits the clue word, and chooses words from the top, stopping when they reach k words or when they choose a word that is not ours, whichever comes first.

Suppose we and our teammate order words by cosine similarities from the same embedding space. Then choosing the optimal clue is just a search over the clue vocabulary for the word with the longest starting run of our team's words in the list sorted by decreasing similarity. In this case, our teammate chooses only the words that we intend, and never chooses a word that is not ours. The search is a matter of enumerating the vocabulary, and its solution is usually not unique.

However, in practice, our perceived similarities are related to but different from our teammate's, which changes the algorithm by which we find the optimal clue. The problem becomes probabilistic rather than deterministic, and we now have to account for the possibility of our teammate choosing a neutral, opponent, or assassin word. This paper describes one algorithm for finding an optimal clue under these conditions.

2 Setup

Let A be the set of our team's board words and B the set of all other board words. For any fixed clue word, let z_x be its similarity to board word x . Throughout this paper, i indexes A and w indexes B .

In order to construct an algorithm, we first define rewards — suppose selecting a word of A is worth 1, and selecting $w \in B$ costs $c_w > 0$. We want to choose the clue word and number k , with $1 \leq k \leq K$ for a fixed maximum K , that maximizes the expected reward.

To model deviations in the guesser's perceived similarities of the words from ours, we assume the guesser perceives board word x 's similarity with the clue word as $\hat{z}_x = z_x + \varepsilon_x$, where the ε_x are independent $\mathcal{N}(0, \sigma^2)$ for some free parameter σ . We also assume the guesser selects unrevealed words in decreasing order of $\hat{z}$, stopping after k selections or upon selecting a word of B , whichever comes first.

Let

$$T = \max_{w \in B} \hat{z}_w, \quad W = \arg \max_{w \in B} \hat{z}_w, \quad N = \#\{i \in A : \hat{z}_i > T\}.$$

In words, W is the non-team word with the highest similarity from the guesser, T is the guesser's similarity of that word, and N is the number of team words that the

guesser ranks higher than W .

We know the guesser reveals $\min(k, N)$ words of A , and selects a word of B (that word being W) if and only if $N < k$. Note that no word of B other than W can be selected. Thus, our goal is to maximize $G(k) - P(k)$, where

$$G(k) = \mathbb{E}[\min(k, N)], \quad P(k) = \mathbb{E}[c_W \mathbf{1}\{N < k\}].$$

3 Derivation

Neither G nor P has a closed form. Both are expectations over all $|A| + |B|$ noise variables at once, and the maximum of non-identical normals has no elementary distribution, so the aim is not to solve them exactly but to reduce them far enough to evaluate cheaply.

We take far enough to mean the following: every quantity is built from φ and Φ , the standard normal PDF and CDF, by finitely many sums and products over A and B , with a single one-dimensional integral in t left over. Treating φ and Φ as known is the convention that treats $\exp$ and $\sqrt{\cdot}$ as known: any standard library computes them to machine precision. The remaining integral is evaluated numerically in Section 4, and what the derivation buys is that one dimension replaces the $|A| + |B|$ we started with.

We start with $G(k)$. Since $\min(k, N) = \sum_{j=1}^k \mathbf{1}\{N \geq j\}$, taking expectations and conditioning on T gives

$$G(k) = \sum_{j=1}^k \int \Pr(N \geq j \mid T = t) dF_T(t).$$

To evaluate this, we need the distribution of T and the distribution of N given $T = t$.

For the first, let φ and Φ be the PDF and CDF of a standard normal, so that the PDF and CDF of $\hat{z}_w \sim \mathcal{N}(z_w, \sigma^2)$ are

$$f_{\hat{z}_w}(t) = \sigma^{-1} \varphi\left(\frac{t - z_w}{\sigma}\right), \quad F_{\hat{z}_w}(t) = \Phi\left(\frac{t - z_w}{\sigma}\right).$$

Since T is a maximum of independent variables,

$$F_T(t) = \prod_{w \in B} F_{\hat{z}_w}(t) = \prod_{w \in B} \Phi\left(\frac{t - z_w}{\sigma}\right).$$

For the second, note that T is a function of $(\varepsilon_w)_{w \in B}$ only, which is independent of $(\varepsilon_i)_{i \in A}$. Thus conditioning on $T = t$ does not change the distribution of the $\hat{z}_i$, and each $i \in A$ independently satisfies $\hat{z}_i > t$ with probability

$$p_i(t) = \Phi\left(\frac{z_i - t}{\sigma}\right).$$

Thus, given $T = t$, N is a sum of $|A|$ independent Bernoulli variables, the i -th with success probability $p_i(t)$.

Now we move on to $P(k)$. Note that if we fix $T = t$, then N is solely a function of $(\varepsilon_i)_{i \in A}$, whereas W is solely a function of $(\varepsilon_w)_{w \in B}$. Thus, c_W and $\mathbf{1}\{N < k\}$ are conditionally independent given T , so

$$P(k) = \int \Pr(N < k \mid T = t) \bar{c}(t) dF_T(t), \quad \bar{c}(t) = \mathbb{E}[c_W \mid T = t].$$

We have already derived the distribution of T and the first factor, so we just need $\bar{c}(t)$, which requires $\Pr(W = w \mid T = t)$.

Writing g_w for the joint density of $W = w$ and T at t ,

$$\Pr(W = w \mid T = t) = \frac{g_w(t)}{f_T(t)}.$$

$W = w$ and $T = t$ means $\hat{z}_w = t$ and $\hat{z}_v < t$ for every other $v \in B$. The $\hat{z}$ are independent, so

$$g_w(t) = f_{\hat{z}_w}(t) \prod_{v \neq w} F_{\hat{z}_v}(t).$$

The events $\{W = w\}$, $w \in B$, are disjoint and exhaustive, so

$$f_T(t) = \sum_{w \in B} g_w(t).$$

Since $F_T(t) = \prod_{v \in B} F_{\hat{z}_v}(t)$, the product over $v \neq w$ is $F_T(t)/F_{\hat{z}_w}(t)$. Set

$$h_w(t) = \frac{f_{\hat{z}_w}(t)}{F_{\hat{z}_w}(t)} = \frac{\sigma^{-1} \varphi((t - z_w)/\sigma)}{\Phi((t - z_w)/\sigma)},$$

so that $g_w(t) = F_T(t) h_w(t)$ and $f_T(t) = F_T(t) \sum_v h_v(t)$. The $F_T(t)$ cancels in the ratio:

$$\Pr(W = w \mid T = t) = \frac{h_w(t)}{\sum_v h_v(t)}, \quad \bar{c}(t) = \frac{\sum_w c_w h_w(t)}{\sum_w h_w(t)}.$$

4 Numerical evaluation

Both G and P have the form $\int \psi(t) dF_T(t)$ for a bounded ψ :

$$\psi_G(t) = \sum_{j=1}^k \Pr(N \geq j \mid T = t), \quad \psi_P(t) = \Pr(N < k \mid T = t) \bar{c}(t).$$

Both are evaluated by numerical integration. Partition an interval $[t_0, t_M]$ into cells $[t_m, t_{m+1}]$, take ψ at the midpoint $\bar{t}_m = \frac{1}{2}(t_m + t_{m+1})$ of each, and weight it by the probability that T lands in that cell:

$$u_m = F_T(t_{m+1}) - F_T(t_m), \quad \int \psi(t) dF_T(t) \approx \sum_m \psi(\bar{t}_m) u_m.$$

The u_m are exact, being differences of the product in Section 3, so the only error is the variation of ψ within a cell.

Take t_0 below every z_w and t_M above every z_w , each by several σ , so that $F_T(t_0)$ and $1 - F_T(t_M)$ are negligible and no mass falls outside the grid. The same grid and the same u_m serve both integrals, so one pass over the cells gives $G(k)$ and $P(k)$ together, and for every $k \leq K$ at once.

Within a cell, $\bar{c}(\bar{t}_m)$ is immediate from its definition, at a cost of $|B|$ terms. The probabilities $\Pr(N \geq j \mid T = \bar{t}_m)$ follow from the standard recursion for a sum of independent Bernoulli variables. Letting $q_r^{(i)}$ be $\Pr(N = r)$ after the first i words of A ,

$$q_0^{(0)} = 1, \quad q_r^{(i)} = q_r^{(i-1)}(1 - p_i(\bar{t}_m)) + q_{r-1}^{(i-1)} p_i(\bar{t}_m),$$

and $\Pr(N \geq j \mid T = \bar{t}_m) = 1 - \sum_{r < j} q_r$. Counts above K are never used, so the recursion may be truncated there, costing $|A|K$ per cell rather than $|A|^2$.

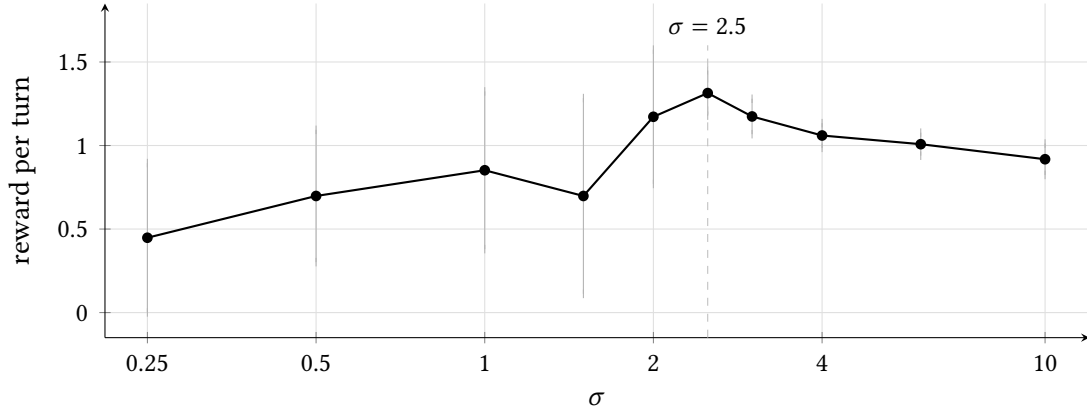
The total is $O(M(|A|K + |B|))$ per clue, with M the number of cells. Every step is a fixed sequence of arithmetic on arrays of the same shape, so the whole clue vocabulary is scored at once rather than one clue at a time.

5 Results

The model above has one free parameter. To choose it, σ was swept over three orders of magnitude and each value was made to play the same 100 positions, drawn from held-out board words and spanning openings through endgames. Each turn was scored as the game scores it: the guesser's ranking is walked, own words are counted until the first word of B , and the reward is +1 per own word with -0.2 , -1 and -10 for a neutral, opponent and assassin word respectively. The guesser was an external language model, not the similarity ranking the spymaster itself uses, so the numbers measure the model against a listener it has no access to.

σ	announced	revealed	gap	reward	s.e.	asn	opp	neu	clean
0.25	3.96	1.55	-2.41	0.448	0.228	5	53	36	6
0.5	3.91	1.60	-2.31	0.698	0.202	3	53	36	8
1.0	3.45	1.74	-1.71	0.852	0.241	4	43	29	24
1.5	2.79	1.72	-1.07	0.698	0.299	7	28	21	44
2.0	2.33	1.69	-0.64	1.172	0.205	3	19	14	64
2.5	1.72	1.41	-0.31	1.314	0.069	0	7	13	80
3.0	1.40	1.22	-0.18	1.174	0.054	0	3	8	89
4.0	1.21	1.10	-0.11	1.060	0.038	0	3	5	92
6.0	1.16	1.05	-0.11	1.008	0.035	0	3	6	91
10.0	1.13	0.99	-0.14	0.918	0.048	0	6	6	88

Per turn, over 100 positions. *announced* is k ; *revealed* is the number of words of A the guesser actually took; the last four columns count how each turn ended.



Realized reward per turn. Bars are 95% confidence intervals; the wide bars at small σ are assassin turns, each costing -10 .

Both limits saturate. As $\sigma \rightarrow 0$ the model treats the guesser as a copy of itself and announces $k = K$ on nearly every board, and as σ grows it announces $k = 1$ and stops responding to further increases. Neither end is optimal, and the reward curve is single-peaked between them.

Note that the number of words revealed and the reward earned are maximized at different σ . Revealed words peak at $\sigma = 1.0$ with 1.74 per turn, where reward is only 0.852; reward peaks at $\sigma = 2.5$ with 1.314, where 1.41 words are revealed. The difference is entirely risk. At $\sigma = 1.0$ the guesser reaches a word of B on 76 of 100 turns and the assassin on 4; at $\sigma = 2.5$ it reaches B on 20 and the assassin on none. Thus the gain from claiming an extra word is real but is more than repaid, and an objective fitted on revealed words alone would choose a value that loses games.

The assassin is what makes the left half of the curve hard to measure. It is worth -10 , so a handful of assassin turns dominate the variance: the standard error is 0.2 to 0.3 for $\sigma \leq 2.0$ and 0.035 to 0.069 for $\sigma \geq 2.5$, where no assassin was reached at all. Comparing paired by board, $\sigma = 2.5$ beats every other value tested at the 5% level except $\sigma = 2.0$ ($+0.14 \pm 0.21$) and $\sigma = 1.0$ ($+0.46 \pm 0.49$), both of which are indistinguishable only because their own assassin losses are so noisy.

Note also that the gap between announced and revealed is negative everywhere and closes monotonically, reaching -0.11 at $\sigma = 4$. The model therefore remains optimistic at its own optimum, announcing 1.72 where 1.41 is delivered. Calibrating G and maximizing reward are different objectives: only the ordering of $G(k) - P(k)$ across clues and k affects the choice made, so a bias shared by every candidate leaves the argmax unchanged. Fitting σ to close the gap would report $\sigma \approx 4$ and cost 0.25 reward per turn.

We therefore take $\sigma = 2.5$. The positions used here are disjoint from the evaluation boards, so the value was not chosen on the boards it is later reported on.

Here the z_x are similarities standardized per clue, c_w are the costs of the words to avoid, and σ is the

guesser's per-word perturbation.